Раздел 1. Цель работы и задачи

Цель работы - реализовать спроектированную в практической работе № 7 онтологию предиктивного обслуживания в виде графа знаний, наполнить его данными из реляционного хранилища (практические работы № 5-6) и проверить пригодность графа для запросов и анализа.

Задачи работы:

1. выбрать инструмент реализации графа знаний и обосновать выбор;
2. спроектировать структуру графа (типы узлов и рёбер) на основе онтологии;
3. установить инструмент и настроить соединение с хранилищем;
4. импортировать данные из PostgreSQL в граф;
5. выполнить тестовые запросы к графу знаний;
6. визуализировать фрагмент графа и сохранить изображение.

Раздел 2. Выбор инструмента и обоснование

Рассмотрены четыре инструмента из методических указаний (таблица 1).

Таблица 1 - Сравнение инструментов реализации графа знаний

Инструмент	Модель / язык	Плюсы для проекта	Минусы для проекта
Neo4j	Property graph / Cypher	Браузер с визуализацией, транзакции, атрибуты рёбер	Отдельный сервер (Docker или установка); онтологию OWL нужно переводить в метки вручную
Apache Jena / TDB	RDF / SPARQL	Стандарты W3C, OWL-вывод, хранение на диске	Java-сервер Fuseki, сложнее в установке и освоении
NetworkX	Граф в памяти / Python API	Алгоритмы на графах, визуализация, без сервера	Нет языка запросов и персистентности, нет связи со схемой онтологии
RDFLib	RDF / SPARQL / Python API	Онтология лабы 7 используется как есть, SPARQL, сохранение в Turtle, без сервера	Нет GUI, медленнее серверных СУБД на больших графах

Выбрана комбинация **RDFLib + NetworkX**. Обоснование - по одной строке на аргумент:

- соответствие задаче: онтология лабы 7 описана в OWL/RDF, и в RDFLib граф знаний строится прямо в её терминах - те же классы, свойства и IRI, без ручного перевода в метки Neo4j;
- функциональность: SPARQL с путями свойств и агрегатами для предметных запросов, NetworkX - для поиска кратчайших путей, степеней узлов и визуализации;
- доступность: оба инструмента ставятся одной командой pip, не требуют сервера и Docker и работают в Windows так же, как в Linux;
- персистентность: граф сохраняется в стандартный формат Turtle и открывается в Protégé, GraphDB или Apache Jena;
- проверяемость: граф можно проверить ризонером по аксиомам онтологии, как в лабе 7.

Установка и соединение: `pip install rdflib networkx sqlalchemy psycopg2-binary`. Подключение к хранилищу выполняется через SQLAlchemy с теми же параметрами, что и в лабе 6 (переменные окружения DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD). Версии: Python 3.11.15, RDFLib 7.6.0, NetworkX 3.6.1, PostgreSQL 16 (хранилище лабы 6).

Раздел 3. Структура графа: маппинг онтологии

Онтология переносится в граф по правилам методических указаний: класс - тип (метка) узла, индивид - узел, свойство данных - атрибут узла, объектное свойство - типизированное ребро. В RDF меткой узла служит триплет rdf:type, а уникальным идентификатором - IRI, построенный из бизнес-ключа хранилища (таблица 2, аналог таблицы 9.1 методических указаний).

Таблица 2 - Маппинг онтологии на структуру графа (узлы)

Класс онтологии	Метка узла (rdf:type)	Ключевые атрибуты узла	Уникальный идентификатор
Оборудование	pdm:MillingMachine	inventoryNumber, nominalPowerW, powerMinW, powerMaxW	pdm:MILL_01
Класс качества	pdm:QualityClass	classCode, osfLimit, wearStepMin	pdm:Class_{L|M|H}
Цикл инструмента	pdm:ToolLifeCycle	cycleNumber, productsProcessed, wearAtEndMin, endReason	pdm:Cycle_{tool_cycle_no}
Наблюдение	pdm:Observation	udi, 5 показаний датчиков, temperatureDifferenceK, powerW, wearTorque, positionInCycle	pdm:Obs_{udi}
Изделие	pdm:Product	productId	pdm:Product_{product_id}
Событие отказа	pdm:FailureEvent	isPrimary	pdm:FE_{udi}_{код}
Тип отказа	pdm:Mechanical/Thermal/ ElectricalFailureType	failureCode, severityRank	pdm:{TWF|HDF|PWF|OSF|RNF}
Правило	pdm:ProductionRule	ruleCode, conditionText, certaintyFactor	pdm:R_{код}
Прогноз риска	pdm:RiskForecast	riskScore, riskLevel	pdm:RF_{udi}_{код}
Узел, датчик, параметр, вид работ	pdm:Spindle ..., pdm:Sensor ..., pdm:MonitoredParameter, pdm:MaintenanceAction	label, unit, actionName	из онтологии лабы 7

Типы рёбер приведены в таблице 3 (аналог таблицы 9.2 методических указаний).

Таблица 3 - Типы рёбер

Тип ребра	От метки	К метке	Атрибуты ребра	Кратность
observedOn	Observation	MillingMachine	-	Многие к одному
processesProduct	Observation	Product	-	Один к одному
hasQualityClass	Product	QualityClass	-	Многие к одному
usesToolCycle	Observation	ToolLifeCycle	positionInCycle (на узле Observation)	Многие к одному
cycleOnEquipment	ToolLifeCycle	MillingMachine	-	Многие к одному
nextObservation	Observation	Observation	-	Один к одному
hasFailureEvent / occurredDuring	Observation	FailureEvent	-	Один ко многим
hasFailureType	FailureEvent	FailureType	isPrimary (на узле события)	Многие к одному
concernsObservation	RiskForecast	Observation	riskScore, riskLevel (на узле прогноза)	Многие к одному
generatedBy	RiskForecast	ProductionRule	-	Многие к одному
predictsFailureType	RiskForecast	FailureType	-	Многие к одному
detects / usesParameter	ProductionRule	FailureType / MonitoredParameter	-	Многие ко многим

Тип ребра	От метки	К метке	Атрибуты ребра	Кратность
affectsComponent / recommendsAction	FailureType	Component / MaintenanceAction	-	Многие ко многим
hasComponent, installedOn, measuredBy, derivedFrom	Equipment, Sensor, Parameter	Component, Parameter	-	Из онтологии

Атрибуты рёбер в RDF задаются через промежуточный узел (реификацию). В графе такую роль уже играют события отказа и прогнозы риска. Событие - это «ребро» наблюдение - тип отказа с атрибутом isPrimary. Прогноз - «ребро» наблюдение - тип отказа с атрибутами riskScore и riskLevel и ссылкой на сработавшее правило. Это соответствует онтологии лабы 7, где событие и прогноз - самостоятельные понятия. Структура графа с фактическим числом узлов и рёбер каждого типа показана на рисунке 1.

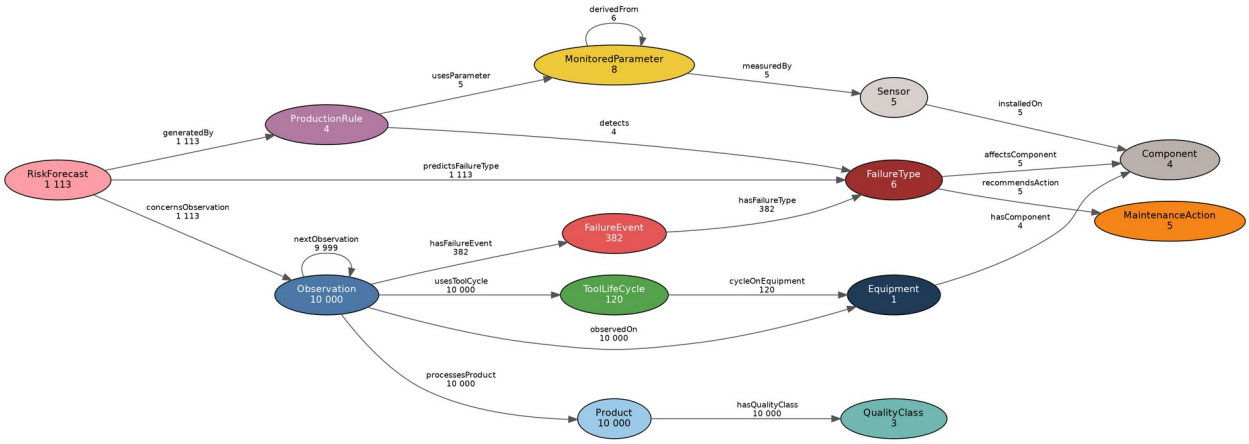


Рисунок 1 - Структура графа знаний: типы узлов и рёбер с количеством

Раздел 4. Импорт данных из хранилища

4.1. Извлечение данных

Программа build_graph.py сначала загружает онтологию лабы 7: схему и справочные индивиды. Примеры наблюдений из лабы 7 (21 индивидов) отбрасываются, потому что граф наполняется полными данными из хранилища. Затем выполняются шесть SQL-запросов к схеме dwh (таблица 4).

Таблица 4 - Источники узлов и рёбер в хранилище

Запрос	Таблицы хранилища	Порождает	Строк
equipment	dim_equipment	атрибуты узла станка	1
quality_class	dim_quality_class	атрибуты узлов классов (SET)	3

Запрос	Таблицы хранилища	Порождает	Строк
tool_cycle	dim_tool	узлы ToolLifeCycle, рёбра cycleOnEquipment	120
observation	fact_observation + 3 измерения	узлы Observation и Product, рёбра observedOn, processesProduct, hasQualityClass, usesToolCycle, nextObservation	10 000
failure_event	fact_failure_event + dim_failure_type	узлы FailureEvent, рёбра hasFailureEvent, hasFailureType	382
forecast	fact_observation + пороги из dim_equipment, dim_quality_class	узлы RiskForecast, рёбра generatedBy, concernsObservation, predictsFailureType	1 113

Прогнозы риска вычисляются запросом: условия правил проверяются по порогам, хранящимся в измерениях. Правило HDF применяется в нестрогой форме (≤ 8,6 К), как рекомендовано по итогам лабы 6.

```
SELECT f.udi, r.code
FROM dwh.fact_observation f
JOIN dwh.dim_equipment e USING (equipment_key)
JOIN dwh.dim_quality_class q USING (class_key)
CROSS JOIN LATERAL (VALUES
    ('HDF', f.temp_diff_k <= e.hdf_temp_diff_min_k AND f.rot_speed_rpm < e.hdf_speed_max_rpm),
    ('PWF', f.power_w < e.power_min_w OR f.power_w > e.power_max_w),
    ('OSF', f.wear_torque > q.osf_limit),
    ('TWF', f.tool_wear_min BETWEEN e.twf_wear_min_min AND e.twf_wear_max_min)
) AS r(code, fired)
WHERE r.fired
```

4.2. Создание узлов и рёбер

Уникальность узлов обеспечивается так же, как операцией MERGE в Cypher. Каждый узел получает IRI из бизнес-ключа (pdm:Obs_161, pdm:Cycle_2), а RDF-граф - это множество триплетов: повторное добавление того же триплета его не дублирует. Для атрибутов используется g.set - аналог SET: старое значение заменяется новым. Фрагмент кода:

```
for row in data['observation'].itertuples():
    obs, prod = PDM[f'Obs_{row.udi}'], PDM[f'Product_{row.product_id}']
    g.add((obs, RDF.type, PDM.Observation))            # MERGE (o:Observation {udi})
    g.set((obs, PDM.udi, integer(row.udi)))            # SET o.udi = ...
    g.set((obs, PDM.torqueNm, dec(row.torque_nm)))
    ...
    g.add((obs, PDM.observedOn, eq_iri[row.inventory_no]))           # ребро
    g.add((obs, PDM.usesToolCycle, PDM[f'Cycle_{row.tool_cycle_no}']))
    g.add((obs, PDM.processesProduct, prod))
    g.add((prod, PDM.hasQualityClass, PDM[f'Class_{row.class_code}']))
    if prev is not None:
        g.add((prev, PDM.nextObservation, obs))
    prev = obs

for row in data['failure_event'].itertuples():
    obs, fe = PDM[f'Obs_{row.udi}'], PDM[f'FE_{row.udi}_{row.failure_code}']
    g.add((fe, RDF.type, PDM.FailureEvent))
    g.add((fe, PDM.hasFailureType, FT[row.failure_code]))
    g.set((fe, PDM.isPrimary, Literal(bool(row.is_primary))))
    g.add((obs, PDM.hasFailureEvent, fe))
```

Полный текст программы приведён в приложении А. Импорт выполняется за 6,5 с, извлечение из хранилища - за 1,1 с. Граф содержит 211

865 RDF-триплетов. Чтобы проверить уникальность, импорт выполнен повторно на том же графе: число триплетов осталось равным 211 865, то есть ни один узел и ни одно ребро не продублированы. Граф сохранён в файл results/knowledge_graph.ttl.gz (440 КБ).

4.3. Количество узлов и рёбер

Количество узлов по типам приведено в таблице 5, рёбер - в таблице 6.

Таблица 5 - Количество узлов по типам

Тип узла	Узлов	Источник
Observation	10 000	fact_observation
Product	10 000	fact_observation.product_id
RiskForecast	1 113	вычисляется по правилам
FailureEvent	382	fact_failure_event
ToolLifeCycle	120	dim_tool
MonitoredParameter	8	онтология
FailureType	6	онтология (= dim_failure_type)
MaintenanceAction	5	онтология
Sensor (4 подкласса)	5	онтология
Component (4 подкласса)	4	онтология
ProductionRule	4	онтология
QualityClass	3	онтология + атрибуты из dim_quality_class
MillingMachine	1	онтология + атрибуты из dim_equipment
Итого	21 651	

Таблица 6 - Количество рёбер по типам

Тип ребра	Рёбер	Тип ребра	Рёбер
observedOn	10 000	hasFailureType	382
processesProduct	10 000	cycleOnEquipment	120
hasQualityClass	10 000	derivedFrom	6
usesToolCycle	10 000	installedOn	5
nextObservation	9 999	measuredBy	5
generatedBy	1 113	usesParameter	5
concernsObservation	1 113	affectsComponent	5
predictsFailureType	1 113	recommendsAction	5
hasFailureEvent	382	hasComponent	4
occurredDuring	382	detects	4
Итого	54 643		

Число узлов и рёбер согласовано с хранилищем: 10 000 наблюдений и изделий, 382 события отказа, 120 циклов инструмента; рёбер nextObservation

на одно меньше, чем наблюдений. Структурные проверки по аксиомам онтологии (таблица 7) нарушений не выявили.

Таблица 7 - Проверка графа по аксиомам онтологии (SPARQL)

Проверка	Нарушений
наблюдение без изделия	0
наблюдение с >1 циклом	0
изделие с >1 классом	0
событие без типа	0
прогноз без правила	0
уровень риска вне перечисления	0

Раздел 5. Тестовые запросы

Выполнены пять запросов (программа queries.py): три обязательных по методическим указаниям и два запроса предметной области. Запросы к содержимому графа написаны на SPARQL, алгоритмы на графах (кратчайший путь, степень узла) выполнены в NetworkX на проекции графа: 21 651 узлов-индивидов и 54 643 рёбер.

5.1. Запрос 1. Все связи заданного узла

Найти все рёбра наблюдения 161 (отказ по перегрузке) в обоих направлениях с типом соседа:

```
SELECT ?direction ?relation ?neighbour ?type WHERE {
  { pdm:Obs_161 ?relation ?neighbour . BIND("->" AS ?direction) }
  UNION
  { ?neighbour ?relation pdm:Obs_161 . BIND("<-" AS ?direction) }
  ?relation a owl:ObjectProperty .
  ?neighbour a ?type . FILTER(?type != owl:NamedIndividual)
} ORDER BY ?direction ?relation
```

Таблица 8 - Результат запроса 1: связи узла pdm:Obs_161

Направление	Ребро	Соседний узел	Тип соседа
исходящее	hasFailureEvent	FE_161_OSF	FailureEvent
исходящее	nextObservation	Obs_162	Observation
исходящее	observedOn	MILL_01	MillingMachine
исходящее	processesProduct	Product_L47340	Product
исходящее	usesToolCycle	Cycle_2	ToolLifeCycle
входящее	concernsObservation	RF_161_OSF	RiskForecast
входящее	concernsObservation	RF_161_TWF	RiskForecast
входящее	nextObservation	Obs_160	Observation
входящее	occurredDuring	FE_161_OSF	FailureEvent

Узел имеет 9 связей. По ним видно, где, над каким изделием и в каком цикле инструмента выполнено наблюдение, какой отказ в нём произошёл, какие правила на нём сработали и какие наблюдения были до и после. На наблюдении сработали два правила: OSF, подтверждённое отказом, и TWF (износ 216 мин в зоне правила). Время выполнения - 149 мс.

5.2. Запрос 2. Кратчайший путь между двумя узлами

Найти кратчайший путь от датчика температуры процесса (S_ProcTemp) до события отказа по теплоотводу в наблюдении 3237 (FE_3237_HDF). Поиск выполнен поиском в ширину (BFS, nx.shortest_path) по неориентированной проекции графа, поскольку граф невзвешенный. Найденный путь длиной 4 рёбер (таблица 9) проходит через станок: датчик установлен на шпинделе станка, на котором выполнено наблюдение.

Таблица 9 - Результат запроса 2: кратчайший путь

Шаг	От узла	Ребро	К узлу
1	S_ProcTemp	-installedOn->	Spindle_01
2	Spindle_01	<-hasComponent-	MILL_01
3	MILL_01	<-observedOn-	Obs_3237
4	Obs_3237	-hasFailureEvent->	FE_3237_HDF

Такой путь верен, но объясняет мало. Если исключить «структурные» рёбра (наблюдение - станок, станок - узел, цикл, порядок наблюдений) и искать путь только по рёбрам базы знаний, получается объяснение, как датчик связан с отказом (таблица 10).

Таблица 10 - Кратчайший путь по рёбрам базы знаний

Шаг	От узла	Ребро	К узлу
1	S_ProcTemp	<-measuredBy-	P_ProcTemp
2	P_ProcTemp	<-derivedFrom-	P_TempDiff
3	P_TempDiff	<-usesParameter-	R_HDF
4	R_HDF	-detects->	HDF
5	HDF	<-hasFailureType-	FE_3237_HDF

Цепочка читается так: датчик измеряет температуру процесса; из неё вычисляется разность температур; разность используется правилом R_HDF; правило распознаёт отказ HDF; событие 3237 имеет этот тип. Тот же путь подтверждён SPARQL-запросом с путями свойств (результат: true):

```
ASK { pdm:FE_3237_HDF pdm:hasFailureType ?ft .
      ?rule pdm:detects ?ft ; pdm:usesParameter/pdm:derivedFrom*/pdm:measuredBy pdm:S_ProcTemp
. }
```

5.3. Запрос 3. Узлы с наибольшим числом связей

Десять узлов с наибольшей степенью (число рёбер без учёта направления) приведены в таблице 11.

Таблица 11 - Результат запроса 3: топ-10 узлов по степени

№	Узел	Тип	Степень
1	MILL_01	MillingMachine	10 124
2	Class_L	QualityClass	6 000
3	Class_M	QualityClass	2 997
4	Class_H	QualityClass	1 003
5	TWF	MechanicalFailureType	839
6	R_TWF	ProductionRule	792
7	HDF	ThermalFailureType	248
8	OSF	MechanicalFailureType	200
9	PWF	ElectricalFailureType	193
10	R_HDF	ProductionRule	133

Наибольшую степень имеют станок (10 000 наблюдений, 120 циклов и 4 узла) и классы качества: степени классов совпадают с распределением изделий 6000 / 2997 / 1003 из лабы 2. Среди типов отказа лидирует TWF, и почти все его связи - прогнозы правила R_TWF: 790 срабатываний на 46 отказов. Высокая степень правила при малом числе подтверждений - структурный признак «шумного» правила, видимый без расчёта метрик.

5.4. Запрос 4. Запросы предметной области

4a. Качество продукционных правил. Для каждого правила подсчитаны срабатывания и подтверждённые срабатывания - те, после которых в том же наблюдении произошёл отказ предсказанного типа. Полнота - доля отказов типа, которым соответствует прогноз (таблица 12).

```
SELECT ?rule (COUNT(DISTINCT ?rf) AS ?fired) (COUNT(DISTINCT ?fe) AS ?confirmed) WHERE {
  ?rf pdm:generatedBy ?r ; pdm:predictsFailureType ?ft ; pdm:concernsObservation ?obs .
  ?r pdm:ruleCode ?rule .
  OPTIONAL { ?obs pdm:hasFailureEvent ?fe . ?fe pdm:hasFailureType ?ft . }
} GROUP BY ?rule ORDER BY ?rule
```

Таблица 12 - Результат запроса 4а: точность и полнота правил по графу знаний

Тип	Сработало	Подтверждено	Точность, %	Отказов	С прогнозом	Полнота, %
HDF	130	115	88,5	115	115	100,0
PWF	95	95	100,0	95	95	100,0
OSF	98	98	100,0	98	98	100,0
TWF	790	43	5,4	46	43	93,5
RNF	-	-	-	19	0	0,0
UNKNOWN	-	-	-	9	0	0,0

Результаты, полученные обходом графа, совпадают с расчётами лаб 2 и 6. Это подтверждает корректность импорта: правила PWF и OSF точны и полны, правило HDF в нестрогой форме даёт полноту 100 % при точности 88,5 %, правило TWF - полноту 93,5 % при точности 5,4 %. Отказы RNF и UNKNOWN не прогнозируются: для них нет правил (лаба 2, п. 4.6).

4б. Отказы по механизму, узлу и рекомендуемой работе. Запрос проходит путь «событие - тип отказа - класс механизма» и «тип отказа - узел - вид работ». Эти знания есть только в онтологии, в хранилище их нет.

```
SELECT ?mechanism ?component ?action (COUNT(?fe) AS ?failures) WHERE {
  ?fe pdm:hasFailureType ?ft .
  ?ft a ?mclass . ?mclass rdfs:subClassOf pdm:FailureType ; rdfs:label ?mechanism .
  ?ft pdm:affectsComponent ?c ; pdm:recommendsAction ?a .
  ?c rdfs:label ?component . ?a pdm:actionName ?action .
  FILTER(lang(?mechanism) = "ru" && lang(?component) = "ru")
} GROUP BY ?mechanism ?component ?action ORDER BY DESC(?failures)
```

Таблица 13 - Результат запроса 4б

Механизм	Поражаемый узел	Рекомендуемая работа	Отказов
Тепловой тип отказа	Система охлаждения MILL-01	Проверка системы охлаждения	115
Механический тип отказа	Шпиндель станка MILL-01	Снижение подачи, внеочередной осмотр	98
Механический тип отказа	Режущий инструмент MILL-01	Снижение подачи, внеочередной осмотр	98
Электрический тип отказа	Привод станка MILL-01	Диагностика привода	95
Механический тип отказа	Режущий инструмент MILL-01	Плановая замена инструмента	46

Чаще всего страдают режущий инструмент (TWF и OSF - 144 отказа с учётом того, что перегрузка поражает и шпиндель) и система охлаждения (115). Для планирования ремонтов это означает, что запас режущего

инструмента и проверка системы охлаждения - приоритетные статьи регламента.

4в. Упреждение правила TWF. Для каждого цикла инструмента, завершившегося отказом по износу, найдено, за сколько изделий до отказа правило TWF впервые сработало в том же цикле. Запрос использует путь «событие - наблюдение - цикл - другие наблюдения цикла - прогнозы»:

```
SELECT ?cycle ?failPos (MIN(?pos) AS ?firstAlertPos) WHERE {
  ?fe pdm:hasFailureType pdm:TWF ; pdm:occurredDuring ?fobs .
  ?fobs pdm:usesToolCycle ?c ; pdm:positionInCycle ?failPos .
  ?c pdm:cycleNumber ?cycle .
  ?rf pdm:generatedBy pdm:R_TWF ; pdm:concernsObservation ?obs .
  ?obs pdm:usesToolCycle ?c ; pdm:positionInCycle ?pos .
} GROUP BY ?cycle ?failPos ORDER BY ?cycle
```

Результат: в 45 циклах из 46 правило сработало до отказа или в момент отказа. Среднее упреждение - 6,1 изделия (от 0 до 20); в 1 циклах правило сработало только на изделии, где произошёл отказ. При шаге износа 2-5 мин на изделие это 12-30 мин работы станка - мало для планового ремонта. Порог TWF в 200 мин следует снизить, чтобы получить предупреждение заранее. Такой вывод возможен только при обходе связей наблюдение - цикл - прогноз, то есть на графе.

Раздел 6. Визуализация фрагмента графа

Фрагмент графа (24 узла, 43 ребра) построен программой visualize.py: NetworkX выделяет подграф вокруг наблюдений 160-162 в цикле инструмента № 2, matplotlib строит рисунок. Разные типы узлов показаны разными цветами (рисунок 2); изображение сохранено в файл img/subgraph.png.

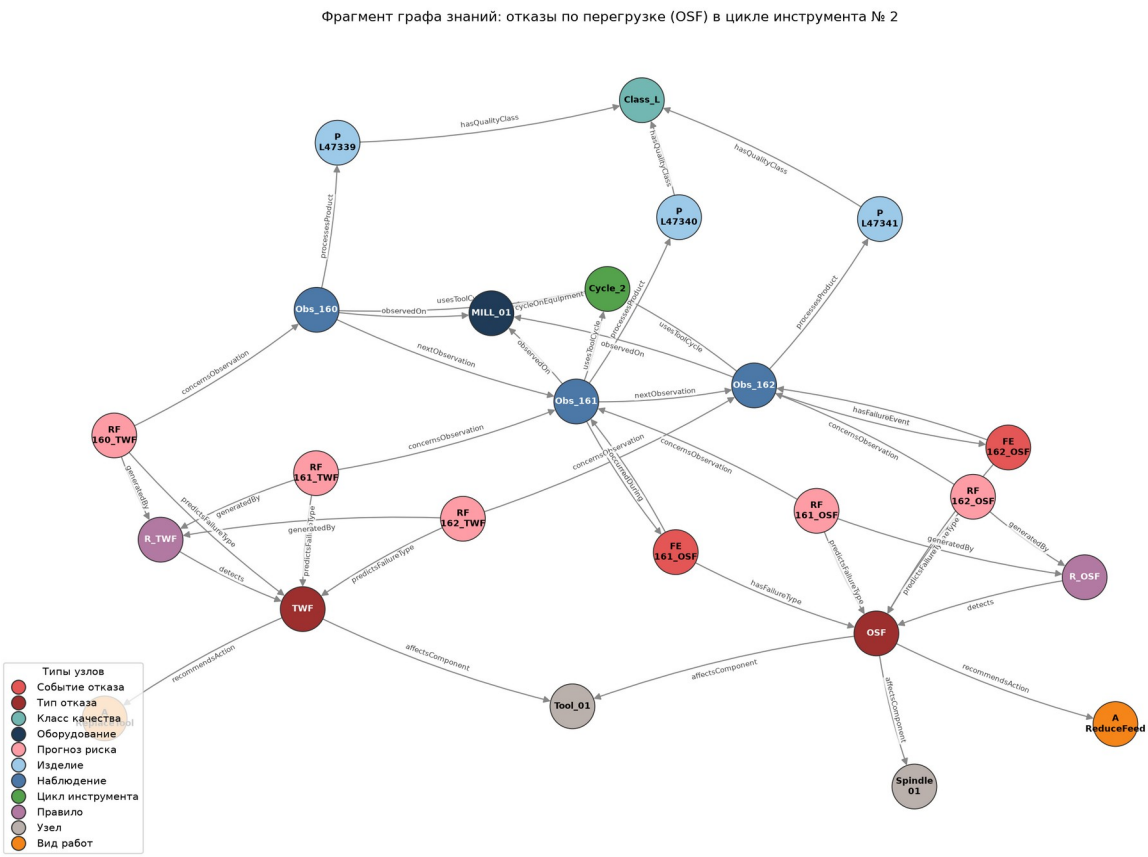


Рисунок 2 - Фрагмент графа знаний: наблюдения 160-162, отказы по перегрузке и связанные знания

На фрагменте видна логика предиктивного обслуживания. Наблюдения 161 и 162 содержат события отказа OSF, и для каждого есть прогноз правила R_OSF - прогнозы подтверждены. Тип OSF связан с поражаемыми узлами (шпиндель и инструмент) и рекомендуемой работой «снижение подачи». Одновременно на всех трёх наблюдениях сработало правило R_TWF: износ превысил 200 мин, но отказа по износу не было - это ложные срабатывания, о которых говорит запрос 4а.

Раздел 7. Выводы

Онтология лабы 7 реализована в виде графа знаний на RDFLib и наполнена данными из хранилища PostgreSQL. Граф содержит 21 651 узлов 13 типов, 54 643 рёбер 20 типов и 211 865 RDF-триплетов. Узлы идентифицируются IRI из бизнес-ключей хранилища, поэтому повторный импорт не создаёт дублей (аналог MERGE). Проверки по аксиомам онтологии нарушений не выявили.

Выполнены пять тестовых запросов: связи узла, кратчайший путь (BFS), топ-10 по степени и два запроса предметной области. Граф воспроизвёл точность и полноту правил из лаб 2 и 6 и дал результаты, недоступные в хранилище без онтологии. Он объясняет связь датчика с отказом через цепочку параметр - правило - тип отказа, распределяет отказы по механизмам и узлам с рекомендуемыми работами и оценивает упреждение правила TWF - в среднем 6,1 изделия, что мало для планирования ремонта.

Трудности и их решения:

- атрибуты рёбер в RDF не задаются напрямую: роль рёбер с атрибутами выполняют узлы событий отказа и прогнозов риска, что соответствует онтологии;
- RDFLib не поддерживает EXISTS внутри агрегатных функций SPARQL; запросы переписаны через OPTIONAL и COUNT(DISTINCT);
- кратчайший путь по всему графу проходит через узел-«хаб» (станок) и мало объясняет; для объяснения поиск ограничен рёбрами базы знаний;
- RDFLib работает в памяти и медленнее серверных СУБД: импорт занимает около 6 с, сложные запросы - секунды. Для 2 млн наблюдений (НФТ № 6 лабы 1) потребуется серверное RDF-хранилище (Apache Jena TDB, GraphDB) или Neo4j; формат Turtle позволяет перенести граф без изменений.

Список использованных источников

1. Добров Б. В. Онтологии и тезаурусы: модели, инструменты, приложения / Б. В. Добров, В. В. Иванов, Н. В. Лукашевич, В. Д. Соловьев. - Москва : URSS, 2013. - ISBN 978-5-9963-0007-5.

2. Гаврилова Т. А. Базы знаний интеллектуальных систем / Т. А. Гаврилова, В. Ф. Хорошевский. - Санкт-Петербург : Питер, 2000. - 384 с. - ISBN 5-272-00071-4.
3. Осипов Г. С. Методы искусственного интеллекта / Г. С. Осипов. - Москва : Физматлит, 2011. - 296 с. - ISBN 978-5-9221-1323-6.
4. Рассел С. Искусственный интеллект: современный подход / С. Рассел, П. Норвиг ; пер. с англ. - 2-е изд. - Москва : Вильямс, 2006. - 1408 с. - ISBN 5-8459-0887-6.
5. SPARQL 1.1 Query Language : W3C Recommendation. - URL: https://www.w3.org/TR/sparql11-query/ (дата обращения: 26.10.2026).
6. RDFLib documentation : [официальная документация]. - URL: https://rdflib.readthedocs.io/ (дата обращения: 26.10.2026).
7. NetworkX documentation : [официальная документация]. - URL: https://networkx.org/documentation/ (дата обращения: 26.10.2026).

Приложение А. Листинг программы построения графа

```
"""Построение графа знаний из хранилища PostgreSQL (лабораторная работа 8, вариант 10).

1. Загружается онтология лабы 7 (схема и справочные индивиды: станок, узлы, датчики,
   параметры, типы отказов, правила, виды работ). Примеры наблюдений из лабы 7 не
   загружаются - вместо них граф наполняется полными данными из хранилища.
2. Из схемы dwh (лабы 5-6) извлекаются данные и создаются узлы и рёбра по маппингу.
3. Граф сохраняется в results/knowledge_graph.ttl.gz; статистика - в results/build.json.

Уникальность узлов: каждому узлу соответствует IRI, построенный из бизнес-ключа
(pdm:Obs_161, pdm:Cycle_2 ...). RDF-граф - множество триплетов, поэтому повторное
добавление того же узла или ребра его не дублирует - аналог MERGE в Cypher.

Параметры подключения - как в лабе 6 (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD).
Запуск: python build_graph.py
"""
import gzip
import json
import os
import time
from decimal import Decimal
from pathlib import Path

import pandas as pd
from rdflib import Graph, Literal, Namespace
from rdflib.namespace import OWL, RDF, XSD
from sqlalchemy import create_engine

HERE = Path(__file__).resolve().parent
ONTO = HERE.parent / 'Лаба_7' / 'ontology' / 'pdm.ttl'
RES = HERE / 'results'
RES.mkdir(exist_ok=True)
PDM = Namespace('http://example.org/pdm#')

URL = 'postgresql+psycopg2://{u}:{p}@{h}:{port}/{db}'.format(
    u=os.getenv('DB_USER', 'postgres'), p=os.getenv('DB_PASSWORD', 'mysecretpassword'),
    h=os.getenv('DB_HOST', 'localhost'), port=os.getenv('DB_PORT', '5432'),
    db=os.getenv('DB_NAME', 'ai_project'))

# Классы, экземпляры которых берутся из хранилища (примеры из лабы 7 отбрасываются)
DATA_CLASSES = [PDM.Observation, PDM.Product, PDM.ToolLifeCycle, PDM.FailureEvent,
PDM.RiskForecast]

# --- SQL-запросы к хранилищу: какой узел порождает каждая таблица -----------------------
SQL = {
    'equipment': 'SELECT equipment_key, inventory_no, nominal_power_w, power_min_w,
power_max_w '
                 'FROM dwh.dim_equipment',
    'quality_class': 'SELECT class_code, osf_limit, wear_step_min FROM dwh.dim_quality_class',
    'tool_cycle': 'SELECT tool_cycle_no, products_processed, wear_at_end_min, end_reason '
                  'FROM dwh.dim_tool ORDER BY tool_cycle_no',
    'observation': 'SELECT f.udi, f.product_id, q.class_code, e.inventory_no, t.tool_cycle_no,
f.cycle_pos, '
                   'f.air_temp_k, f.process_temp_k, f.rot_speed_rpm, f.torque_nm,
f.tool_wear_min, '
                   'f.temp_diff_k, f.power_w, f.wear_torque '
                   'FROM dwh.fact_observation f '
                   'JOIN dwh.dim_quality_class q USING (class_key) '
                   'JOIN dwh.dim_equipment e USING (equipment_key) '
                   'JOIN dwh.dim_tool t USING (tool_key) ORDER BY f.udi',
    'failure_event': 'SELECT e.udi, ft.failure_code, e.is_primary '
                     'FROM dwh.fact_failure_event e JOIN dwh.dim_failure_type ft USING
(failure_type_key)',
    # Прогнозы: правила вычисляются по порогам из измерений хранилища. Правило HDF - в
    # нестрогой форме (<=), как рекомендовано по итогам лабы 6 (п. 4.4).
    'forecast': """
        SELECT f.udi, r.code
        FROM dwh.fact_observation f
        JOIN dwh.dim_equipment e USING (equipment_key)
        JOIN dwh.dim_quality_class q USING (class_key)
        CROSS JOIN LATERAL (VALUES
            ('HDF', f.temp_diff_k <= e.hdf_temp_diff_min_k AND f.rot_speed_rpm <
e.hdf_speed_max_rpm),
            ('PWF', f.power_w < e.power_min_w OR f.power_w > e.power_max_w),
            ('OSF', f.wear_torque > q.osf_limit),
            ('TWF', f.tool_wear_min BETWEEN e.twf_wear_min_min AND e.twf_wear_max_min)
        ) AS r(code, fired)
        WHERE r.fired
```

```
        ORDER BY f.udi, r.code""",
}
FT = {'TWF': PDM.TWF, 'HDF': PDM.HDF, 'PWF': PDM.PWF, 'OSF': PDM.OSF, 'RNF': PDM.RNF,
      'UNKNOWN': PDM.UnknownFailure}
RULE = {'HDF': PDM.R_HDF, 'PWF': PDM.R_PWF, 'OSF': PDM.R_OSF, 'TWF': PDM.R_TWF}
CF = {'HDF': Decimal('1.0'), 'PWF': Decimal('1.0'), 'OSF': Decimal('1.0'), 'TWF':
Decimal('0.06')}


def dec(x):
    return Literal(Decimal(str(x)), datatype=XSD.decimal)


def integer(x):
    return Literal(int(x), datatype=XSD.integer)


def load_ontology():
    g = Graph()
    g.parse(ONTO)
    drop = {s for c in DATA_CLASSES for s in g.subjects(RDF.type, c)}
    for s in drop:
        g.remove((s, None, None))
        g.remove((None, None, s))
    # списки owl:AllDifferent для удалённых примеров тоже удаляются
    for b in list(g.subjects(RDF.type, OWL.AllDifferent)):
        members = list(g.collection(g.value(b, OWL.distinctMembers)))
        if not members:
            g.remove((b, None, None))
    return g, len(drop)


def import_data(g, data):
    """Создание узлов и рёбер. Повторный вызов не создаёт дублей (идемпотентность)."""
    eq_iri = {}
    for row in data['equipment'].itertuples():
        node = next(g.subjects(PDM.inventoryNumber, Literal(row.inventory_no)), None) \
            or PDM[f'Equipment_{row.equipment_key}']
        eq_iri[row.inventory_no] = node
        g.add((node, RDF.type, PDM.MillingMachine))
        g.set((node, PDM.nominalPowerW, dec(row.nominal_power_w)))
        g.set((node, PDM.powerMinW, dec(row.power_min_w)))
        g.set((node, PDM.powerMaxW, dec(row.power_max_w)))

    for row in data['quality_class'].itertuples():            # SET: значения из хранилища
        node = PDM[f'Class_{row.class_code}']
        g.set((node, PDM.osfLimit, integer(row.osf_limit)))
        g.set((node, PDM.wearStepMin, integer(row.wear_step_min)))

    for row in data['tool_cycle'].itertuples():
        node = PDM[f'Cycle_{row.tool_cycle_no}']
        g.add((node, RDF.type, PDM.ToolLifeCycle))
        g.add((node, RDF.type, OWL.NamedIndividual))
        g.set((node, PDM.cycleNumber, Literal(int(row.tool_cycle_no),
datatype=XSD.positiveInteger)))
        g.set((node, PDM.productsProcessed, Literal(int(row.products_processed),
datatype=XSD.positiveInteger)))
        g.set((node, PDM.wearAtEndMin, integer(row.wear_at_end_min)))
        g.set((node, PDM.endReason, Literal(row.end_reason)))
        g.add((node, PDM.cycleOnEquipment, next(iter(eq_iri.values()))))

    prev = None
    for row in data['observation'].itertuples():
        obs, prod = PDM[f'Obs_{row.udi}'], PDM[f'Product_{row.product_id}']
        g.add((obs, RDF.type, PDM.Observation))
        g.add((obs, RDF.type, OWL.NamedIndividual))
        g.set((obs, PDM.udi, integer(row.udi)))
        g.set((obs, PDM.positionInCycle, Literal(int(row.cycle_pos),
datatype=XSD.positiveInteger)))
        g.set((obs, PDM.airTemperatureK, dec(row.air_temp_k)))
        g.set((obs, PDM.processTemperatureK, dec(row.process_temp_k)))
        g.set((obs, PDM.rotationalSpeedRpm, integer(row.rot_speed_rpm)))
        g.set((obs, PDM.torqueNm, dec(row.torque_nm)))
        g.set((obs, PDM.toolWearMin, integer(row.tool_wear_min)))
        g.set((obs, PDM.temperatureDifferenceK, dec(row.temp_diff_k)))
        g.set((obs, PDM.powerW, dec(row.power_w)))
        g.set((obs, PDM.wearTorque, dec(row.wear_torque)))
        g.add((obs, PDM.observedOn, eq_iri[row.inventory_no]))
        g.add((obs, PDM.usesToolCycle, PDM[f'Cycle_{row.tool_cycle_no}']))
        g.add((obs, PDM.processesProduct, prod))
```

```
        g.add((prod, RDF.type, PDM.Product))
        g.add((prod, RDF.type, OWL.NamedIndividual))
        g.set((prod, PDM.productId, Literal(row.product_id)))
        g.add((prod, PDM.hasQualityClass, PDM[f'Class_{row.class_code}']))
        if prev is not None:
            g.add((prev, PDM.nextObservation, obs))
        prev = obs

    for row in data['failure_event'].itertuples():
        obs, fe = PDM[f'Obs_{row.udi}'], PDM[f'FE_{row.udi}_{row.failure_code}']
        g.add((fe, RDF.type, PDM.FailureEvent))
        g.add((fe, RDF.type, OWL.NamedIndividual))
        g.add((fe, PDM.hasFailureType, FT[row.failure_code]))
        g.set((fe, PDM.isPrimary, Literal(bool(row.is_primary))))
        g.add((obs, PDM.hasFailureEvent, fe))
        g.add((fe, PDM.occurredDuring, obs))

    for row in data['forecast'].itertuples():
        rf = PDM[f'RF_{row.udi}_{row.code}']
        g.add((rf, RDF.type, PDM.RiskForecast))
        g.add((rf, RDF.type, OWL.NamedIndividual))
        g.add((rf, PDM.generatedBy, RULE[row.code]))
        g.add((rf, PDM.concernsObservation, PDM[f'Obs_{row.udi}']))
        g.add((rf, PDM.predictsFailureType, FT[row.code]))
        g.set((rf, PDM.riskScore, Literal(CF[row.code], datatype=XSD.decimal)))
        g.set((rf, PDM.riskLevel, Literal('high' if CF[row.code] >= Decimal('0.5') else
'low')))


def main():
    t0 = time.perf_counter()
    engine = create_engine(URL)
    with engine.connect() as conn:
        data = {k: pd.read_sql(q, conn) for k, q in SQL.items()}
    t_extract = time.perf_counter() - t0
    g, dropped = load_ontology()
    onto_triples = len(g)
    # атрибут события «основной подтип» - свойство данных, добавляемое к онтологии лабы 7
    g.add((PDM.isPrimary, RDF.type, OWL.DatatypeProperty))
    g.add((PDM.isPrimary, RDF.type, OWL.FunctionalProperty))

    t = time.perf_counter()
    import_data(g, data)
    t_import = time.perf_counter() - t
    n1 = len(g)
    import_data(g, data)                        # повторный импорт: дублей быть не должно
    n2 = len(g)

    nodes = {c: len(set(g.subjects(RDF.type, PDM[c]))) for c in [
        'MillingMachine', 'Component', 'Spindle', 'Drive', 'CoolingSystem', 'CuttingTool',
        'TemperatureSensor', 'SpeedSensor', 'TorqueSensor', 'WearCounter',
'MonitoredParameter',
        'QualityClass', 'Product', 'ToolLifeCycle', 'Observation', 'FailureEvent',
        'MechanicalFailureType', 'ThermalFailureType', 'ElectricalFailureType',
'RandomFailureType',
        'FailureType', 'ProductionRule', 'RiskForecast', 'MaintenanceAction']}
    edges = {p: len(list(g.subject_objects(PDM[p]))) for p in [
        'observedOn', 'processesProduct', 'hasQualityClass', 'usesToolCycle',
'cycleOnEquipment',
        'nextObservation', 'hasFailureEvent', 'occurredDuring', 'hasFailureType',
'generatedBy',
        'concernsObservation', 'predictsFailureType', 'hasComponent', 'installedOn',
'measuredBy',
        'derivedFrom', 'detects', 'usesParameter', 'affectsComponent', 'recommendsAction']}
    with gzip.open(RES / 'knowledge_graph.ttl.gz', 'wb') as f:
        f.write(g.serialize(format='turtle').encode('utf-8'))
    stats = {'rows': {k: len(v) for k, v in data.items()}, 'nodes': nodes, 'edges': edges,
             'triples_ontology': onto_triples, 'triples': n1, 'triples_after_reimport': n2,
             'dropped_examples': dropped, 'time_extract_s': t_extract, 'time_import_s':
t_import,
             'time_total_s': time.perf_counter() - t0,
             'file_kb': (RES / 'knowledge_graph.ttl.gz').stat().st_size / 1024}
    json.dump(stats, open(RES / 'build.json', 'w', encoding='utf-8'), ensure_ascii=False,
indent=1)
    print(f'Строк из хранилища: {stats["rows"]}')
    print(f'Триплетов: онтология {onto_triples}, после импорта {n1}, после повторного импорта
{n2}')
    print('Узлы:', nodes)
    print('Рёбра:', edges)
    print(f'Время: извлечение {t_extract:.2f} с, импорт {t_import:.2f} с')
```

```
if __name__ == '__main__':
    main()
```